

GUÍA DE INTEGRACIÓN TÉCNICA

Mastersoft ERP ↔ HubSpot CRM

HubSpot → Mastersoft ERP

Flujo 1 · Creación / Actualización de Clientes vía Webhook

Mastersoft ERP → HubSpot

Flujo 2A · Sincronización de Compañías y Contactos

Mastersoft ERP → HubSpot

Flujo 2B · Sincronización Diaria de Cuenta Corriente (Batch)

Audiencia: Equipo de Desarrollo — Mastersoft

Versión: 3.0

Fecha: Abril 2026

FLUJO 1 — HubSpot → Mastersoft ERP

Recepción de Webhook y Creación de Clientes

1. Estructura del Payload (Webhook)

HubSpot envía un **array JSON** al endpoint configurado en Mastersoft cada vez que la propiedad `crear_en_erp` cambia a `true` en un objeto de tipo **Compañía**.

1.1 Ejemplo de Payload

```
[
  {
    "eventId": 3727832674,
    "subscriptionId": 5950199,
    "portalId": 51121026,
    "appId": 34671955,
    "occurredAt": 1774617646684,
    "subscriptionType": "contact.propertyChange",
    "attemptNumber": 5,
    "objectId": 205319884583,
    "propertyName": "crear_en_erp",
    "propertyValue": "true",
    "changeSource": "AUTOMATION_PLATFORM"
  }
]
```

1.2 Campos Relevantes

Campo	Tipo	Descripción
objectId	number	ID de la Compañía en HubSpot. NOTA: Es de Compañía, no de contacto.
propertyName	string	Debe ser "crear_en_erp".
propertyValue	string	El proceso SOLO se ejecuta si su valor es "true".
attemptNumber	number	Número de intento. Útil para lógica de reintentos.

IMPORTANTE

El campo `objectId` corresponde al ID de la Compañía (Company) en HubSpot, NO al ID de un contacto. Todas las consultas posteriores deben realizarse sobre el endpoint de Companies.

2. Flujo de Ejecución — Backend Mastersoft

Al recibir el webhook, el backend de Mastersoft debe ejecutar los siguientes pasos en orden:

2.1 Validación de Firma (Security)

Antes de procesar cualquier dato, el servidor **DEBE validar** que la petición proviene de HubSpot usando HMAC-SHA256.

```
// Pseudocódigo - Node.js
const crypto = require('crypto');
function validateHubSpotSignature(req) {
  const signature = req.headers['x-hubspot-signature-v3'];
  const timestamp = req.headers['x-hubspot-request-timestamp'];
  const source =
`${req.method}${req.originalUrl}${req.rawBody}${timestamp}`;
  const expected = crypto
    .createHmac('sha256', process.env.HUBSPOT_CLIENT_SECRET)
    .update(source).digest('base64');
  if (!crypto.timingSafeEqual(Buffer.from(signature),
    Buffer.from(expected)))
    throw new Error('Firma inválida - HTTP 401');
}
```

 Ref: [HubSpot — Validating Webhook Request Signatures](#)

2.2 Validación del Payload

- `propertyValue === "true"` — Si es diferente, responder HTTP 200 sin procesar.
- `propertyName === "crear_en_erp"` — Confirmar que es la propiedad correcta.
- `objectId` presente y numérico — Es el ID de Compañía a procesar.

2.3 Obtención de Datos de la Compañía

Con el `objectId` realizar un GET a HubSpot para recuperar todas las propiedades necesarias.

```
GET https://api.hubapi.com/crm/v3/objects/companies/{companyId}
  ?properties=name,domain,phone,address,city,state,country,zip,industry
Authorization: Bearer {PRIVATE_APP_TOKEN}
```

2.4 Obtención de Contactos Asociados

Recuperar todos los contactos vinculados y sus datos básicos:

```
// Paso A – IDs de contactos asociados
GET
https://api.hubapi.com/crm/v3/objects/companies/{companyId}/associations/contacts

// Paso B – Datos en batch (más eficiente para múltiples contactos)
POST https://api.hubapi.com/crm/v3/objects/contacts/batch/read
Body: { "inputs":[{"id":"101"}, {"id":"102"}],
"properties":["firstname", "lastname", "email", "phone", "jobtitle"]} }
```

2.5 Sincronización con Mastersoft

1. **Verificar existencia:** Buscar cliente por dominio o identificador en Mastersoft.
2. **Crear o actualizar:** Según resultado, invocar el endpoint del ERP.
3. **Registrar contactos:** Crear las personas de contacto asociadas.
4. **Confirmar:** Guardar el ID de Mastersoft y escribirlo de vuelta en HubSpot si aplica.

3. Manejo de Errores

Si cualquier paso falla, el sistema debe **crear automáticamente una Nota en HubSpot** asociada a la Compañía para que el equipo de CRM tome acción correctiva.

3.1 Política de Reintentos

- **attemptNumber < 3:** Responder HTTP 500. HubSpot reintentará con backoff exponencial.
- **attemptNumber ≥ 3:** Considerar fallido definitivamente. Crear Nota de error y responder HTTP 200.

3.2 Creación de Nota de Error

```
POST https://api.hubapi.com/crm/v3/objects/notes
{
  "properties": {
    "hs_note_body": "ERROR: La compañía '[Nombre]' (ID: [objectId]) no se migró a Mastersoft. Motivo: [descripción]. Fecha: [ISO timestamp]",
    "hs_timestamp": "[Unix timestamp ms]"
  },
  "associations": [{
    "to": { "id": "{companyId}" },
    "types": [{ "associationCategory": "HUBSPOT_DEFINED", "associationTypeId": 190 }]
  }]
}
```

4. Referencia de Endpoints — Flujo 1

Paso	Endpoint	Método	Propósito
1. Seguridad	Validar header X-HubSpot-Signature-v3	LOCAL	Verificar autenticidad del webhook
2. Compañía	GET /crm/v3/objects/companies/{id}	GET	Obtener propiedades de la empresa
3. Asociaciones	GET /crm/v3/objects/companies/{id}/associations/contacts	GET	Listar IDs de contactos vinculados
4. Contactos	POST /crm/v3/objects/contacts/batch/read	POST	Datos de múltiples contactos en batch
5. ERP	POST/PUT Mastersoft endpoint (interno)	POST	Crear o actualizar cliente en ERP
6. Nota Error	POST /crm/v3/objects/notes	POST	Registrar error en HubSpot si falla

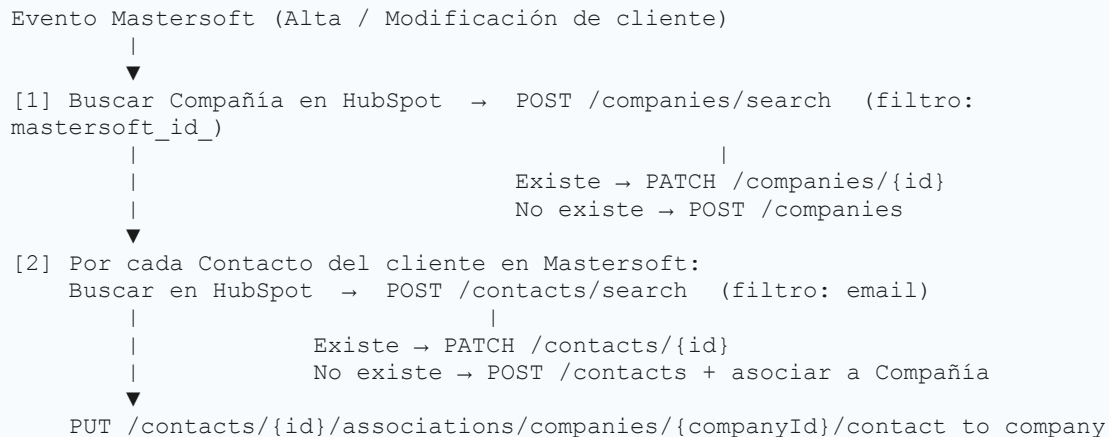
FLUJO 2A — Mastersoft ERP → HubSpot

Creación y Actualización de Compañías y Contactos

5. Creación / Actualización de Compañías y Contactos

Se dispara desde un **evento en Mastersoft ERP** (alta o modificación de cliente). El objetivo es crear o actualizar la **Compañía** y sus **Contactos** asociados en HubSpot.

5.1 Diagrama del Flujo



5.2 Pasos de Ejecución

Paso 1 — Buscar Compañía por mastersoft_id_

Antes de crear, verificar si la compañía ya existe usando la propiedad única `mastersoft_id_`. Si `results.total > 0` → PATCH. Si `results.total === 0` → POST.

```
POST https://api.hubapi.com/crm/v3/objects/companies/search
{
  "filterGroups": [{ "filters": [{
    "propertyName": "mastersoft_id_",
    "operator":      "EQ",
    "value":         "{mastersoft_id_del_cliente}"
  }]}],
  "properties": ["name", "mastersoft_id_", "cuitcuil", "nro_cliente"]
}
```

Paso 2 — Crear (POST) o Actualizar (PATCH) Compañía

Crear: POST <https://api.hubapi.com/crm/v3/objects/companies>

Actualizar: PATCH

https://api.hubapi.com/crm/v3/objects/companies/{hubspot_company_id}

```
// Body (igual para POST y PATCH)
{
  "properties": {
    // — Datos principales
    "name": "{Razon_Social}",
    "nombre_fantasia": "{Nombre_Fantasia}",
    "cuitcuil": "{CUIT_CUIL}",
    "nro_cliente": "{Nro_Cliente}",
    "mastersoft_id": "{ID_Mastersoft}",
    // — Domicilio Fiscal
    "adress": "{Calle}", "puerta": "{Puerta}",
    "city": "{Localidad}", "zip": "{CP}",
    "state": "{Provincia}", "Country": "{Pais}",
    "zona_vta": "{Zona_Vta}",
    // — Datos Comerciales
    "vendedor": "{Vendedor}",
    "responsable_de_cuenta": "{Responsable}",
    "lista_de_precios": "{Lista_Precios}",
    "condicion_de_venta": "{Condicion_Venta}",
    "dias_para_deuda": {Dias_Deuda},
    "limite_de_credito": {Limite_Credito},
    "categoria_cliente": "{Categoria}",
    // — Domicilios de Entrega (hasta 3)
    "direccion_1_domicilio": "{Dir1}", "direccion_1_cp": {CP1},
    "direccion_1_localidad": "{Loc1}", "direccion_1_provincia": "{Prov1}"
    // ... repetir para direccion_2_* y direccion_3_*
  }
}
```

Paso 3 — Buscar Contacto por email y Crear o Actualizar

Por cada contacto, buscar por email como clave de deduplicación.

```
// 3A — Buscar contacto
POST https://api.hubapi.com/crm/v3/objects/contacts/search
Body: { "filterGroups": [{ "filters": [{ "propertyName": "email",
"operator": "EQ", "value": "{email}" }]}] }

// 3B — Crear o actualizar
POST https://api.hubapi.com/crm/v3/objects/contacts // contacto
nuevo
PATCH https://api.hubapi.com/crm/v3/objects/contacts/{id} // contacto
existente

{ "properties": { "firstname": "{Nombre}", "email": "{Email}", "phone":
"{Teléfono}", "sector": "{Sector}" } }
```

Paso 4 — Asociar Contacto a Compañía (solo en creación)

Si el contacto fue **creado** (POST), vincularlo. Si fue **actualizado**, la asociación ya existe.

```
PUT https://api.hubapi.com/crm/v3/objects/contacts/{contactId}/
associations/companies/{companyId}/contact_to_company
```

5.3 Tabla Resumen — Endpoints Flujo 2A

Paso	Endpoint	Método	Propósito
1. Buscar Empresa	POST /crm/v3/objects/companies/search	POST	Deduplicar por mastersoft_id_
2a. Crear Empresa	POST /crm/v3/objects/companies	POST	Nueva empresa no encontrada
2b. Actualizar	PATCH /crm/v3/objects/companies/{id}	PATCH	Empresa ya existe en HubSpot
3a. Buscar Contacto	POST /crm/v3/objects/contacts/search	POST	Deduplicar por email
3b. Crear Contacto	POST /crm/v3/objects/contacts	POST	Nuevo contacto no encontrado
3c. Actualizar	PATCH /crm/v3/objects/contacts/{id}	PATCH	Contacto ya existe en HubSpot
4. Asociar	PUT /contacts/{id}/associations/companies/{id}/...	PUT	Vincular contacto a empresa

5.4 Mapeo de Propiedades — Compañía

Mastersoft (Campo)	Origen (Módulo)	HubSpot (Nombre)	HubSpot (Interno)
Razon Social	Empresa	Company Name	name
Nombre Fantasia	Empresa	Nombre Fantasia	nombre_fantasia
CUIT/CUIL	Empresa	CUIT/CUIL	cuitcuil
NRO CLIENTE	Empresa	NRO CLIENTE	nro_cliente
mastersoft id	Empresa	mastersoft id	mastersoft_id_
Manejo Cuenta Corriente	Empresa	Manejo Cuenta Corriente	manejo_cuenta_corriente
Calle	Domicilio Fiscal	Street Address	adress
Puerta	Domicilio Fiscal	Puerta	puerta
Localidad	Domicilio Fiscal	City	city

Mastersoft (Campo)	Origen (Módulo)	HubSpot (Nombre)	HubSpot (Interno)
C.P	Domicilio Fiscal	Postal Code	zip
Provincia	Domicilio Fiscal	State/Region	state
País	Domicilio Fiscal	Country/Region	Country
Zona Vta	Domicilio Fiscal	Zona vta	zona_vta
Vendedor	Datos comerciales	Vendedor	vendedor
Responsable de Cuenta	Datos comerciales	Responsable de cuenta	responsable_de_cuenta
Lista de Precios	Datos comerciales	Lista de precios	lista_de_precios
Condicion de Venta	Datos comerciales	Condicion de venta	condicion_de_venta
Días para Deuda	Datos comerciales	Dias para deuda	dias_para_deuda
Límite de Crédito	Datos comerciales	Limite de credito	limite_de_credito
CATEGORIA CLIENTE	Datos comerciales	Categoria Cliente	categoria_cliente
Dirección 1 Dom.	Domicilios de Entrega	Direccion 1 Domicilio	direccion_1_domicilio
Dirección 1 CP	Domicilios de Entrega	Direccion 1 CP	direccion_1_cp
Dirección 1 Local.	Domicilios de Entrega	Direccion 1 Localidad	direccion_1_localidad
Dirección 2 Dom.	Domicilios de Entrega	Direccion 2 Domicilio	direccion_2_domicilio
Dirección 2 CP	Domicilios de Entrega	Direccion 2 CP	direccion_2_cp
Dirección 2 Local.	Domicilios de Entrega	Direccion 2 Localidad	direccion_2_localidad
Dirección 3 Dom.	Domicilios de Entrega	Direccion 3 Domicilio	direccion_3_domicilio
Dirección 3 CP	Domicilios de Entrega	Direccion 3 CP	direccion_3_cp
Dirección 3 Local.	Domicilios de Entrega	Direccion 3 Localidad	direccion_3_localidad

Contacto:

Mastersoft (Campo)	Origen (Módulo)	HubSpot (Nombre)	HubSpot (Interno)
Nombre	Contacto	First Name	firstname
Sector	Contacto	Sector	sector
Teléfono	Contacto	Phone Number	phone
Email	Contacto	Email	email

FLUJO 2B — Mastersoft ERP → HubSpot

Sincronización Diaria de Cuenta Corriente — Proceso Batch

6. Sincronización Diaria de Cuenta Corriente

¿Qué hace este proceso?

Una vez al día (entre las 7 y 8 AM), Mastersoft consulta el estado de cuenta corriente de todos sus clientes y actualiza la propiedad `manejo_cuenta_corriente` en HubSpot usando un proceso batch. Los clientes con facturas impagas reciben los datos actualizados; los que ya pagaron reciben la propiedad limpia (string vacío).

6.1 Consideraciones de Diseño

Disparador del proceso: El equipo de Mastersoft define la tecnología y mecanismo de disparo (job interno del ERP, cron en servidor propio, tarea programada, etc.). Lo que importa es que el proceso se ejecute una vez al día en el horario establecido.

Fuente de datos: Mastersoft obtiene la lista de clientes y su estado de cuenta corriente desde su propia base de datos o API interna. El equipo de Mastersoft define cómo recuperar esa información.

Estrategia clave: El proceso siempre actualiza **todos** los clientes en cada ejecución, no solo los que tienen deuda. Esto garantiza que los clientes que pagaron queden limpios automáticamente.



**IMPORT
ANTE**

NO procesar solo los clientes con deuda. Procesar siempre el universo completo de clientes activos. De lo contrario, los clientes que pagaron mantendrían datos de deuda desactualizados en HubSpot.

6.2 Diagrama del Flujo Batch

```
Proceso diario disparado por Mastersoft (horario a definir internamente)
  |
  ▼
[1] Mastersoft consulta internamente TODOS los clientes activos
    con su estado de cuenta corriente (facturas vencidas e impagas)
    |
    ▼
[2] Construir lista de actualizaciones para el batch:
    ├── Cliente con deuda → valor: "15/03/2026 --- $125.400,00\n02/04/2026 ---
    $89.200,50"
    └── Cliente sin deuda → valor: "" (limpiar propiedad)
    |
    ▼
[3] Dividir en grupos de 100 registros máximo (límite del batch)
    |
    ▼
[4] POST /crm/v3/objects/companies/batch/update por cada grupo
    |
    ▼
[5] Registrar resultados: OK / errores en log interno de Mastersoft
```

6.3 Formato de la Propiedad manejo_cuenta_corriente

La propiedad es de tipo **Single-line text** en HubSpot. Una línea por factura, separadas con \n:

```
// Formato cuando HAY facturas impagas (una línea por factura):
15/03/2026 --- $125.400,00
02/04/2026 --- $89.200,50
18/04/2026 --- $210.000,00

// Cuando NO hay deuda - limpiar la propiedad:
"" // string vacío
```

```
// Función para construir el valor de la propiedad
function buildCuentaCorriente(facturas) {
  if (!facturas || facturas.length === 0) return '';
  return facturas
    .map(f => `${f.vencimiento} --- ${f.monto.toLocaleString('es-AR', {
      minimumFractionDigits: 2 })}`)
    .join('\n');
}
```

6.4 Implementación del Batch Update

En lugar de hacer un PATCH individual por cliente, usar el endpoint **batch/update** de HubSpot. Esto reduce drásticamente la cantidad de requests y evita problemas de rate limiting.

Paso A — Construir los inputs del batch

Antes de llamar a HubSpot, cada cliente necesita su **hubspot_company_id**. Si el Flujo 2A ya sincronizó los clientes, ese ID debería estar almacenado en Mastersoft. Si no, recuperarlo buscando por `mastersoft_id_`.

```
// Pseudocódigo: construir el array de inputs
const inputs = clientes.map(cliente => ({
  id: cliente.hubspot_company_id, // ID de HubSpot almacenado en Mastersoft
  properties: {
    manejo_cuenta_corriente: buildCuentaCorriente(cliente.facturas_impagas)
    // clientes sin deuda reciben string vacío "" automáticamente
  }
}));
```

Paso B — Ejecutar el Batch Update

```
// El endpoint acepta hasta 100 registros por request
POST https://api.hubapi.com/crm/v3/objects/companies/batch/update

Headers:
  Authorization: Bearer {PRIVATE_APP_TOKEN}
  Content-Type: application/json

Body:
{
  "inputs": [
    {
      "id": "205319884583",
      "properties": {
        "manejo_cuenta_corriente": "15/03/2026 --- $125.400,00\n02/04/2026
--- $89.200,50"
      }
    },
    {
      "id": "205319884601",
      "properties": {
        "manejo_cuenta_corriente": ""
      }
    }
  ]
  // ... hasta 100 registros por request
}
```

Paso C — Paginación y manejo de errores del batch

```
// Pseudocódigo completo del proceso batch
async function syncCuentaCorrienteBatch(clientes) {
  const BATCH_SIZE = 100; // límite de HubSpot
  const DELAY_MS = 200; // pausa entre batches para respetar rate limit
  const errores = [];

  // Dividir en grupos de 100
  for (let i = 0; i < clientes.length; i += BATCH_SIZE) {
    const grupo = clientes.slice(i, i + BATCH_SIZE);
    const inputs = grupo.map(c => ({
      id: c.hubspot_company_id,
      properties: { manejo_cuenta_corriente:
buildCuentaCorriente(c.facturas_impagas) }
    }));

    try {
      const res = await
fetch('https://api.hubapi.com/crm/v3/objects/companies/batch/update', {
        method: 'POST',
        headers: { 'Authorization': `Bearer ${TOKEN}`, 'Content-Type':
'application/json' },
        body: JSON.stringify({ inputs })
      });

      if (!res.ok) {
        const err = await res.json();
        errores.push({ batch: i / BATCH_SIZE + 1, error: err });
      }
    } catch (e) {
      errores.push({ batch: i / BATCH_SIZE + 1, error: e.message });
    }

    await new Promise(r => setTimeout(r, DELAY_MS)); // pausa entre batches
  }

  logger.info(`Sync CC: ${clientes.length} clientes, ${errores.length}
errores`);
  if (errores.length > 0) logger.warn('Errores en batch CC:', errores);
}
```

6.5 Comparativa: Batch vs. Individual

Aspecto	PATCH Individual	Batch Update (recomendado)
Requests por 1.000 clientes	1.000 requests	10 requests (grupos de 100)
Rate limit (100 req/10s)	Requiere delays grandes	Casi sin riesgo de límite
Tiempo estimado (1.000 cl.)	~100 segundos	~2–5 segundos
Manejo de errores	Por registro individual	Por batch (respuesta agrupa errores)
Complejidad de implementación	Baja	Media (paginación)

6.6 Tabla Resumen — Endpoints del Proceso Batch

Paso	Endpoint	Método	Propósito
1. Fuente datos	Consulta interna Mastersoft (DB / API propia)	INTERNO	Lista clientes + facturas impagas
2. Buscar Empresa	POST /crm/v3/objects/companies/search	POST	Recuperar hubspot_company_id si no se tiene
3. Batch Update	POST /crm/v3/objects/companies/batch/update	POST	Actualizar / limpiar manejo_cuenta_corriente

6.7 Consideraciones Adicionales

- **Rate limiting:** Con batch/update se consumen aproximadamente 1 request cada 100 clientes. Agregar un delay de 200ms entre batches como precaución.
- **Horario:** El equipo de Mastersoft define el horario exacto según su infraestructura. Se recomienda ejecutar entre las 7 y 8 AM hora Argentina, antes del inicio de la jornada laboral.
- **Idempotencia:** El proceso es idempotente. Ejecutarlo múltiples veces produce el mismo estado final.
- **Independencia:** Este proceso es completamente independiente del Flujo 1 (webhook) y del Flujo 2A (alta de clientes).
- **hubspot_company_id:** Se recomienda que Mastersoft almacene el ID de HubSpot de cada empresa (devuelto en el POST/PATCH del Flujo 2A) para evitar tener que buscarlo en cada ejecución diaria.
- **Monitoreo:** Registrar: cantidad de clientes procesados, cuántos con deuda, cuántos limpiados, cuántos con error y tiempo total de ejecución.